exiting the function.

It seems exactly what we do with set, right? The only difference is that we use a read lock instead, so the difference must be there – and it actually is.

Only one thread can hold a write lock at a single time, but a read lock can be hold by many threads at the same time – provided no thread is holding a write lock.

When we read from a data structure (or a database etc…) we don't modify it, so if 1, 2, 10 or a million threads are reading from the same resource at the same time, they will always get consistent results (if the invariant is implemented correctly, and we don't have side effects that mutate the resource).

When we write on a resource, we change it, and while the process is ongoing, all reads and writes would make no sense, because they would be based on inconsistent data.

Listing 7.11 LFU Cache get, Java version

```
public Value get(Key key) {

readLock.lock();
                        #1
  try {
    PriorityQueueNode node = this.hashTable.get(key);
    if (node == null) {
      return null;
    } else {
      node.updatePriority(node.getCounter() +
1);                                #2
      return node.getValue();
```